



FINAL PROJECT PROPOSAL

Sign Language Recognition System

A Final-Year Machine-Learning Project Proposal

Riphah International University

Department of RISE MSAI

Course / Subject: Machine Learning

Submitted By — Group Members	Submitted To
1. Azhar Hussain (Group Lead) 2. Rahma 3. Haris Javaid 4. Syed Farakh Abbas	Junaid Khan Course Instructor — Machine Learning

Date of Submission: 11 May 2026

Academic Session 2025-2026 | Confidential — For Academic Evaluation Only

Table of Contents

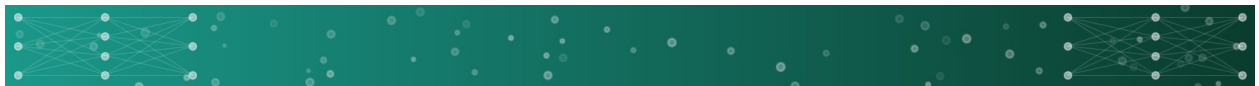
If the entries below do not display page numbers in your viewer, place the cursor in the table and press Ctrl+A then F9 (Word) — “Update Field → Update entire table”.

2. Abstract.....	4
3. Introduction.....	4
4. Problem Statement.....	5
5. Objectives.....	5
6. Scope of the Project.....	6
In Scope.....	6
Out of Scope (Limitations).....	6
7. Literature Review.....	6
7.1 Key Works.....	6
7.2 Comparison of SLR Approaches.....	7
7.3 Datasets & Resources.....	7
8. Proposed Methodology.....	8
8.1 System Architecture.....	8
8.2 Step-by-Step Workflow.....	8
8.3 Workflow Flowchart.....	9
8.4 Algorithms, Models & Frameworks.....	10
9. Technologies and Tools.....	10
10. Dataset Information.....	11
10.1 Candidate Datasets.....	11
10.2 Data Pre-processing.....	11
10.3 Training / Validation / Testing Split.....	11
10.4 Data Augmentation.....	11
11. Expected Outcomes.....	12
12. Innovation / Uniqueness.....	12
13. Feasibility Analysis.....	13
13.1 Technical Feasibility.....	13
13.2 Cost Analysis.....	13
13.3 Hardware Feasibility & GPU / Resource Limitations.....	13
13.4 Practical Implementation Plan.....	14
14. Hardware and Software Requirements.....	14
15. Future Enhancements.....	14
16. Timeline / Gantt Chart.....	15
17. Conclusion.....	16

18. References.....16

2. Abstract

Deaf and hard-of-hearing people communicate primarily through sign language, but most hearing people do not understand it, creating a daily communication barrier in schools, hospitals, banks and public offices. This project proposes a Sign Language Recognition System that uses a webcam and computer vision to translate hand-sign gestures into on-screen text and synthesised speech in real time. Rather than feeding raw images to a heavy CNN, the system uses Google's MediaPipe Hands to extract 21 three-dimensional hand landmarks per frame; these compact, lighting- and background-robust coordinates are normalised and fed to a lightweight classifier — a small multi-layer perceptron or 1-D CNN for static signs (e.g. the ASL alphabet), and an LSTM/GRU over a short frame window for dynamic gestures (motion-based words). Recognised letters/words are assembled into sentences with an auto-complete helper and spoken aloud via an offline text-to-speech engine. Because MediaPipe is highly optimised and the classifiers are tiny, the entire pipeline runs in real time on an ordinary laptop CPU with a basic webcam — no GPU required. Targeted performance is $\geq 95\%$ accuracy on a held-out set of static alphabet signs and a robust, low-latency live demo, delivered as a desktop application with text and voice output.



Keywords: *Sign language recognition, MediaPipe Hands, hand landmarks, gesture recognition, LSTM, assistive technology, real-time computer vision, text-to-speech, accessibility.*

3. Introduction

Sign languages are full natural languages expressed through hand shapes, movements, orientation, and facial expression. The World Health Organization estimates that over 400 million people worldwide have disabling hearing loss; in Pakistan, Pakistan Sign Language (PSL) serves a large Deaf community. Yet outside specialised settings there are very few interpreters, so routine interactions — a doctor's appointment, a bank transaction, a classroom question — often break down. An automatic system that recognises signs from a camera and renders them as text and speech can act as a always-available, low-cost interpreter for many everyday situations.

Computer-vision approaches to sign recognition have ranged from glove-based sensors and depth cameras (Kinect) to deep CNNs on raw RGB images and 3-D CNNs / I3D on video clips. Image-based CNNs are sensitive to lighting, skin tone, clothing and background, and video models are compute-heavy. A more practical route — and the one this project takes — is landmark-based recognition: a pretrained hand-tracking model (MediaPipe Hands, based on the BlazePalm/hand-landmark networks of Zhang et al., 2020) outputs 21 normalised 3-D keypoints per detected hand at high frame rates on CPU. Classifying these low-dimensional, invariant features needs only a tiny neural network, which is fast to train, fast to run, and far more robust to nuisance variation. Dynamic, motion-based signs are handled by adding a short temporal model (LSTM/GRU) over a sliding window of landmark frames.

The scope is sized for a one-semester, four-member project: focus on the ASL/PSL fingerspelling alphabet plus a curated set of common dynamic words, build a clean real-time UI, and add sentence construction and offline speech output. The emphasis is on a robust, genuinely usable assistive prototype rather than maximising vocabulary size.

4. Problem Statement

Communication between Deaf signers and the hearing majority is obstructed by a chronic shortage of interpreters and the general public's lack of sign-language knowledge. Existing technical aids have drawbacks:

- **Hardware-heavy:** Glove/sensor solutions are expensive, intrusive and not always available.
- **Fragile / costly:** Raw-image CNN recognisers are brittle to lighting, background, skin tone and camera quality, and often need a GPU.
- **Slow:** Video (3-D CNN) models are accurate but too compute-intensive to run live on a typical laptop.
- **Incomplete:** Most demos recognise isolated letters only and stop there — they don't build words/sentences or produce speech.

Formally: given a live webcam stream, detect the signing hand(s), recognise the current static sign (e.g. an alphabet letter) or dynamic gesture (a motion-based word), assemble the recognised tokens into readable text with basic word completion, and output that text both on screen and as synthesised speech — all in real time on commodity, GPU-free hardware. This project delivers that end-to-end assistive pipeline.

5. Objectives

- **Primary objective:** Build a real-time webcam pipeline that detects hands (MediaPipe), recognises static ASL/PSL alphabet signs with $\geq 95\%$ accuracy on a held-out test set, and runs at ≥ 15 FPS on a laptop CPU.
- **Secondary objective:** Add a dynamic-gesture module (LSTM/GRU over a short landmark-frame window) for a curated set of common motion-based signs/words.
- **Secondary objective:** Implement landmark normalisation (translation/scale invariance, optional handedness normalisation) so recognition is robust to position, distance and background.
- **Secondary objective:** Build a sentence constructor with debouncing/temporal smoothing and an auto-complete/word-suggestion helper to turn recognised letters into fluent text.
- **Secondary objective:** Integrate offline text-to-speech (pyttsx3) — with optional online gTTS — to speak the constructed text aloud.
- **Secondary objective:** Deliver a clean desktop UI showing the webcam feed with hand-landmark overlay, the current prediction with confidence, the building sentence, and play/clear controls.
- **Learning objective:** Demonstrate competence in real-time computer vision, feature engineering from landmarks, lightweight classifiers, sequence models (LSTM/GRU), and assistive-application design.

6. Scope of the Project

In Scope

- Real-time recognition of the static ASL/PSL fingerspelling alphabet (A–Z, allowing for J/Z motion) from a webcam.
- A dynamic-gesture module for a curated set ($\approx 10\text{--}30$) of common motion-based words (e.g. hello, thanks, yes, no, please).
- MediaPipe-based hand detection and 21-landmark extraction; landmark normalisation and feature engineering.
- Lightweight classifiers (MLP / 1-D CNN for static; LSTM/GRU for dynamic) trained on public + self-collected data.
- Sentence construction with temporal smoothing/debouncing and an auto-complete helper.
- On-screen text output plus offline text-to-speech; a desktop application UI with landmark overlay and controls.

Out of Scope (Limitations)

- Full continuous sign-language translation with grammar, facial expressions and body pose (a much larger research problem).
- Large-vocabulary word recognition (e.g. full WLASL-2000) — only a curated subset is targeted.
- Two-hand complex co-articulated signs beyond the curated set; signer-independent perfection in all conditions.
- Mobile/embedded deployment (feasible later) and speech-to-sign (the reverse direction).
- Production-grade robustness in extreme lighting, motion blur or heavy occlusion.

7. Literature Review

Sign language recognition (SLR) research spans sensor-based methods, image-based deep learning, video-based temporal models, and — increasingly — skeleton/landmark-based approaches that mirror trends in human-action recognition. The works below justify our landmark-centric, lightweight design.

7.1 Key Works

Early SLR used data gloves and accelerometers, and later depth cameras (Microsoft Kinect) to obtain hand/skeleton data; these are accurate but require special hardware. With deep learning, CNNs were applied directly to RGB hand images (e.g. ASL-alphabet classifiers), and for dynamic signs, 3-D CNNs and Inflated 3D ConvNets (I3D; Carreira & Zisserman, 2017) and CNN-LSTM hybrids modelled motion in video; Li et al. (2020) released WLASL, a large word-level ASL video benchmark, with I3D baselines. In parallel, skeleton-based action recognition (e.g. ST-GCN, Yan et al., 2018) showed that joint coordinates alone carry strong discriminative signal. The enabling technology for our approach is MediaPipe Hands (Zhang et al., 2020): a real-time, on-device hand-tracking pipeline (palm detector + hand-landmark regressor) producing 21 3-D landmarks per hand at high frame rates on CPU/mobile. Building classifiers on these landmarks — MLPs and 1-D CNNs for static poses, LSTM/GRU or lightweight Transformers/GCNs for dynamic sequences — is now a common, efficient SLR recipe; numerous IEEE/Elsevier papers report 95–99 % accuracy on alphabet datasets with

such pipelines. Text-to-speech is handled by standard engines (pyttsx3 offline; Google TTS online).

7.2 Comparison of SLR Approaches

Approach	Input / Features	Strength	Limitation / Cost	Typical Accuracy*
Sensor glove / accelerometer	Bend/flex & motion sensors	Very robust signal	Special hardware; intrusive; costly	high (but impractical)
Depth camera (Kinect) + ML	Depth + skeleton joints	3-D info; lighting-robust	Requires depth sensor	≈ 90–96 %
CNN on raw RGB hand image	Cropped hand image	Simple; no landmark library	Brittle to light/background/skin; often needs GPU	≈ 90–95 %
3-D CNN / I3D on video	Stacked RGB frames	Captures motion well (dynamic signs)	Compute-heavy; not real-time on CPU	≈ 90–96 % (WLASL subsets lower)
MediaPipe landmarks + MLP/1-D CNN ★	21×3 normalised keypoints (static)	Tiny, fast, robust, CPU real-time	Static poses only; depends on landmark quality	≈ 96–99 % (alphabet)
MediaPipe landmarks + LSTM/GRU ★	Landmark sequence over a window (dynamic)	Handles motion; still lightweight	Needs sequence data collection	≈ 95–98 % (curated words)
Skeleton GCN (ST-GCN-style)	Graph over hand/body joints	Strong for complex gestures	More complex; more data	≈ 95–98 %

*Table 3.1: SLR approaches. ★ = methods adopted (MediaPipe landmarks + lightweight static and dynamic classifiers). *Accuracies are indicative ranges reported across alphabet/curated-word datasets; they are not directly comparable across different datasets.*

7.3 Datasets & Resources

Resource	Content	Use Here
ASL Alphabet (Kaggle)	≈ 87 000 images, 29 classes (A–Z + space/del/nothing)	Train/test the static-sign classifier
Sign Language MNIST	≈ 34 600 28×28 images, 24 static letters	Quick prototyping baseline
WLASL (subset)	Word-level ASL videos (2 000 words; we use a small subset)	Optional dynamic-gesture data
Self-collected landmark data	Team members record alphabet + curated words via webcam → save MediaPipe landmarks	Domain-matched data; covers PSL signs not in public sets
MediaPipe Hands, pyttsx3/gTTS, NLTK	Pretrained hand tracker; TTS engines; word lists	Feature extraction, speech output, auto-complete

Table 3.2: Datasets and resources considered.

Research gap addressed: many projects classify isolated ASL letters from images and stop. Fewer deliver a robust landmark-based pipeline that also handles dynamic gestures, builds sentences with smoothing and auto-complete, speaks the output, and

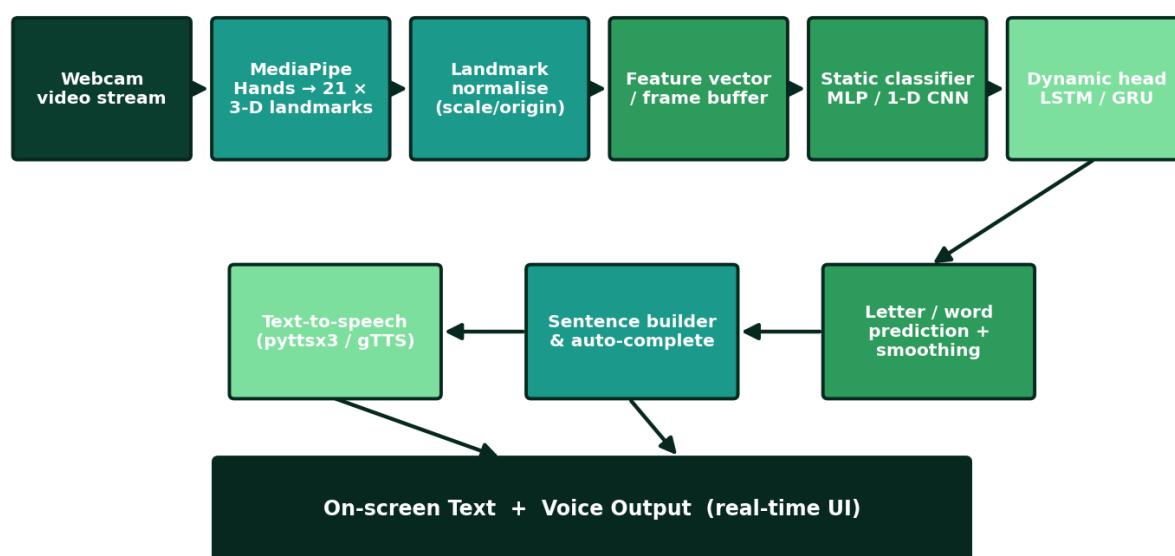
runs in real time on a GPU-free laptop — and fewer still consider Pakistan Sign Language. This project targets that practical, accessibility-focused gap.

8. Proposed Methodology

The methodology comprises a data-collection/feature-engineering stage, a model-training stage (separate static and dynamic models), and a real-time application/integration stage.

8.1 System Architecture

System Architecture — Sign Language Recognition



Pipeline stages flow left → right (top) then converge into the application layer (bottom).

Figure 3.1: End-to-end architecture — webcam → MediaPipe 21-landmark extraction → normalisation → static MLP/1-D CNN and dynamic LSTM/GRU classifiers → sentence builder → on-screen text + text-to-speech.

8.2 Step-by-Step Workflow

1. **Data collection** — Use the ASL Alphabet/Sign-Language-MNIST datasets for static signs and record additional webcam clips (team + volunteers) for PSL signs and dynamic words; for every frame, run MediaPipe Hands and store the 21×3 landmark array (plus handedness) with the label.
2. **Landmark normalisation & features** — Translate landmarks so the wrist is the origin; scale by a reference distance (e.g. wrist-to-middle-finger-MCP) for distance invariance; optionally mirror left hands to a canonical handedness; derive extra features (inter-joint distances/angles). For dynamic signs, stack normalised landmarks over a sliding window (e.g. 20–30 frames).
3. **Static classifier** — Train a small MLP (or 1-D CNN over the 63-value vector) to classify alphabet signs; use dropout and class balancing; this network has only a few thousand-tens-of-thousands of parameters and trains in minutes on CPU.
4. **Dynamic classifier** — Train an LSTM/GRU (1–2 layers) over the landmark-sequence window to classify the curated motion-based words; pad/truncate sequences; apply temporal data augmentation.

5. **Real-time inference** — Capture webcam frames with OpenCV; run MediaPipe; normalise; route to the static model each frame and to the dynamic model on the rolling window; obtain class + confidence.
6. **Temporal smoothing / debouncing** — Stabilise output with a majority vote / confidence threshold over the last k frames so a letter is only 'committed' when held steady; suppress jitter and the no-hand state.
7. **Sentence construction** — Append committed letters/words to a text buffer; map special signs to space/backspace; offer auto-complete word suggestions (frequency dictionary / prefix trie).
8. **Text-to-speech** — On demand (or sentence-end), speak the buffer aloud using pyttsx3 (offline) or gTTS (online); show speaking status in the UI.
9. **UI integration** — Display the webcam feed with the hand-landmark skeleton overlay, the current prediction and confidence bar, the building sentence, FPS, and buttons for speak / clear / static-vs-dynamic mode.
10. **Testing & evaluation** — Measure per-class accuracy and a confusion matrix on held-out data, plus a live user test (recognition rate, latency, FPS) with several signers; document results.

8.3 Workflow Flowchart

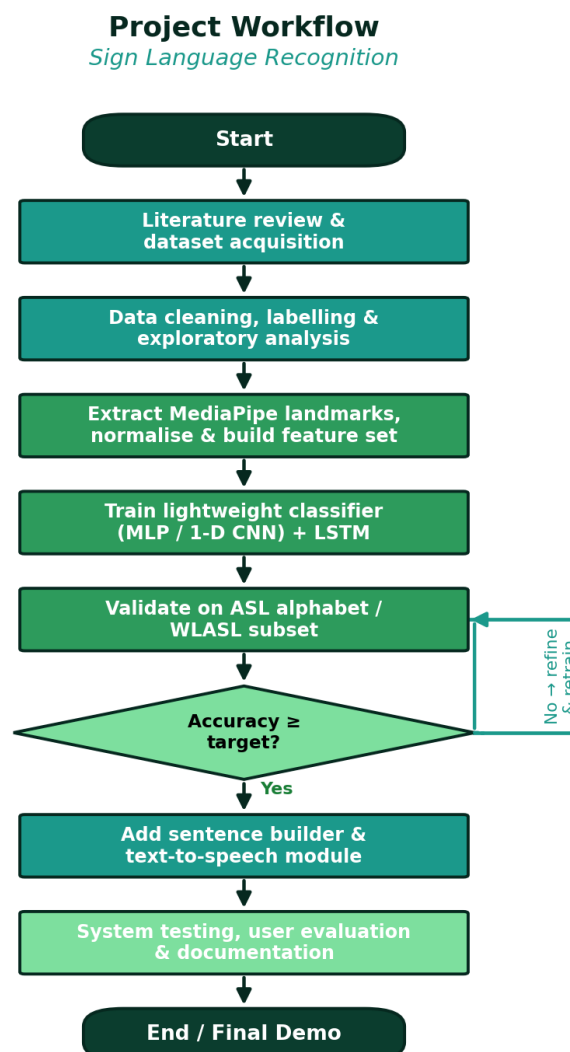


Figure 3.2: Project workflow with the accuracy decision gate and refinement loop.

8.4 Algorithms, Models & Frameworks

- **MediaPipe Hands (pretrained):** Palm detector + hand-landmark model → 21 3-D landmarks per hand, real-time on CPU.
- **Static classifier:** Small MLP (e.g. 63→128→64→#classes) or 1-D CNN over the landmark vector.
- **Dynamic classifier:** 1-2-layer LSTM/GRU over a window of normalised landmark frames (optional lightweight Transformer/GCN).
- **Smoothing:** Sliding-window majority vote + confidence thresholding (debouncing).
- **Text-to-speech:** pyttsx3 (offline, default) or gTTS + playback (online option).
- **Auto-complete:** Frequency-ranked dictionary / prefix trie for word suggestions.
- **Frameworks:** Python, OpenCV, MediaPipe, NumPy, scikit-learn, TensorFlow-Keras or PyTorch (for LSTM), pyttsx3/gTTS, Tkinter/PyQt or Streamlit.

9. Technologies and Tools

Category	Tools / Technologies	Purpose in the Project
Programming language	Python 3.10+	Vision pipeline, model training, UI and integration
Hand tracking	Google MediaPipe (Hands solution)	Pretrained real-time 21-landmark extraction on CPU
Computer vision	OpenCV	Webcam capture, frame handling, drawing landmark overlays
ML / DL frameworks	scikit-learn; TensorFlow-Keras or PyTorch	Static MLP/1-D CNN and dynamic LSTM/GRU models
Numerical	NumPy, pandas	Landmark arrays, feature engineering, dataset handling
Text-to-speech	pyttsx3 (offline); gTTS + playsound (online option)	Speaking the constructed sentences aloud
NLP helper	NLTK / wordfreq, prefix trie	Auto-complete and word suggestions
Application UI	Tkinter / PyQt (alt. Streamlit)	Live feed, prediction display, sentence panel, controls
Visualisation	Matplotlib, seaborn	Confusion matrices, accuracy curves, evaluation plots
Training environment	Local CPU; Google Colab (free) if needed	Tiny models train on CPU; Colab optional for LSTM
Dev tools	VS Code, Jupyter, Git/GitHub	Coding, experiments, version control, collaboration
Dataset sources	ASL Alphabet & Sign-Language-MNIST (Kaggle); WLASL subset; self-collected landmarks	Static training/testing; optional dynamic data; domain-matched PSL data

Table: Technology stack selected for cost-free, low-resource development.

10. Dataset Information

10.1 Candidate Datasets

- **ASL Alphabet (Kaggle):** $\approx 87\,000$ RGB images across 29 classes (A–Z plus space/delete/nothing); the main source for the static-sign classifier (we process images through MediaPipe to obtain landmarks).
- **Sign Language MNIST:** $\approx 34\,600$ grayscale 28×28 images of 24 static letters; a quick baseline/sanity-check dataset.
- **WLASL (subset):** Word-level ASL video clips; a small curated subset (common words) for the dynamic-gesture module.
- **Self-collected landmark dataset:** Team members and volunteers record the alphabet and ≈ 10 – 30 dynamic words via webcam; for each frame MediaPipe landmarks (21×3 + handedness) are saved with the label — this covers PSL signs and matches our deployment conditions.

10.2 Data Pre-processing

- Run MediaPipe Hands on every image/frame; discard frames with no confident hand detection; keep the 21×3 landmark array (and handedness) as the feature record.
- Normalise: subtract the wrist coordinate (translation invariance); divide by a reference bone length (scale invariance); optionally mirror left hands; flatten to a 63-value vector (static) or stack into a $[T \times 63]$ tensor (dynamic).
- Engineer optional extra features: pairwise fingertip distances and joint angles to help the classifier.
- Balance classes; shuffle; for dynamic data, segment continuous recordings into fixed-length windows with overlap; encode labels.

10.3 Training / Validation / Testing Split

Split	Proportion	Purpose
Training	$\approx 70\%$	Fit the static MLP/1-D CNN and the dynamic LSTM/GRU
Validation	$\approx 15\%$	Hyper-parameter tuning, early stopping, model selection
Testing	$\approx 15\%$	Final per-class accuracy and confusion matrix; held-out signers where possible

Table 3.3: Data partitioning. Where enough signers are available, the split is signer-independent so that no signer appears in both training and testing.

10.4 Data Augmentation

Working in landmark space makes augmentation cheap and effective: small random rotations of the landmark cloud (in-plane and slight 3-D tilt), random scaling and translation, mild Gaussian jitter on each coordinate (sensor-noise simulation), random horizontal mirroring (handedness), and — for dynamic gestures — temporal jitter (frame dropping/duplication, speed warping, window shifting). This multiplies the

effective dataset size and improves robustness to camera angle, distance and signing style without collecting more raw video.

11. Expected Outcomes

- A real-time webcam application that overlays hand landmarks and recognises static ASL/PSL alphabet signs with $\geq 95\%$ held-out accuracy at ≥ 15 FPS on a laptop CPU.
- A dynamic-gesture module recognising a curated set of common motion-based words via an LSTM/GRU over landmark sequences.
- Stable output thanks to temporal smoothing/debouncing, with the recognised letters/words assembled into sentences.
- An auto-complete/word-suggestion helper that speeds up sentence construction.
- Offline text-to-speech (pyttsx3) that speaks the constructed text aloud; optional online gTTS.
- A clean desktop UI (live feed + landmark overlay, prediction + confidence, sentence panel, FPS, controls).
- Complete documentation: code repository, data-collection scripts, evaluation report (accuracy, confusion matrix, latency/FPS) and user guide.

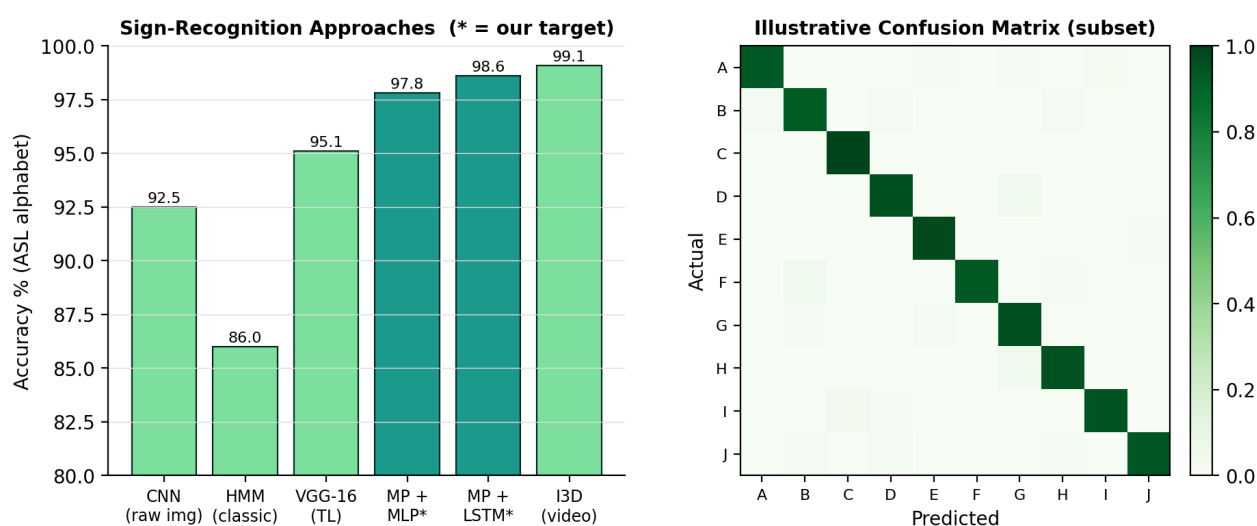


Figure 3.3: (left) Accuracy of representative sign-recognition approaches with our landmark-based targets; (right) an illustrative confusion matrix for a subset of alphabet classes.

12. Innovation / Uniqueness

- **Landmark-first, GPU-free design:** Uses pretrained MediaPipe landmarks + tiny classifiers so the whole system runs in real time on a plain laptop CPU — robust to lighting/background/skin tone.
- **Static + dynamic in one system:** Combines an instantaneous static-sign classifier with an LSTM/GRU for motion-based signs, instead of handling only isolated letters.
- **Full assistive pipeline:** Goes beyond classification to sentence building (with smoothing and auto-complete) and spoken output — a usable communication aid, not just a demo.
- **Pakistan-Sign-Language awareness:** Self-collected data includes PSL signs, addressing the local community and a gap in public datasets.

- **Reproducible & low-cost:** Free datasets and tools, tiny models trainable on CPU, optional Colab — fully reproducible by students with zero budget.
- **Robustness engineering:** Landmark normalisation (translation/scale/handedness invariance) plus landmark-space augmentation make recognition resilient to camera angle, distance and signer style.

13. Feasibility Analysis

13.1 Technical Feasibility

MediaPipe Hands is a free, well-documented, highly optimised library that already solves the hardest part (robust real-time hand landmarking). The classifiers on top are small enough to train on a CPU in minutes (static) or with a short Colab session (dynamic LSTM). OpenCV, scikit-learn/Keras, pytsx3 and Tkinter/PyQt are all mature. Numerous published projects confirm 95–99 % alphabet accuracy with this exact recipe, so the targets are realistic.

13.2 Cost Analysis

Item	Option Chosen	Cost (PKR)
Compute	Local laptop CPU (training & inference); Colab free if needed	0
Datasets	ASL Alphabet / Sign-Language-MNIST / WLASL subset (public) + self-collected	0
Software & libraries	Python, MediaPipe, OpenCV, scikit-learn/Keras, pytsx3 (open-source)	0
Camera	Built-in laptop webcam (or an inexpensive USB webcam)	0–2 500 (optional)
Development machines	Members' existing laptops	0 (already owned)
Demo packaging (optional)	PyInstaller executable / Streamlit Cloud (free)	0
Estimated total	Core project	≈ 0 (PKR 0–2 500 if a USB webcam is bought)

Table 3.4: Indicative cost analysis — essentially zero-budget.

13.3 Hardware Feasibility & GPU / Resource Limitations

- No GPU is required: MediaPipe runs in real time on CPU and the classifiers are tiny; a Colab GPU is only an optional convenience for training the dynamic LSTM.
- Any laptop with a webcam, 4–8 GB RAM and a modern dual/quad-core CPU is sufficient; FPS can be tuned by limiting MediaPipe to one hand and a moderate resolution.
- Resource mitigations: cache extracted landmarks (don't re-run MediaPipe during training), keep models small, use early stopping, and design the UI loop to skip frames gracefully under load.

13.4 Practical Implementation Plan

Weeks 1–4: literature, dataset download, MediaPipe pipeline, landmark extraction & normalisation, baseline static classifier. Weeks 5–8: collect custom data, train/tune the static MLP/1-D CNN to target accuracy, build the dynamic-data recorder. Weeks 9–12: train the dynamic LSTM/GRU, add temporal smoothing, sentence builder, auto-complete and text-to-speech. Weeks 13–16: real-time UI integration, optimisation, user testing with several signers, evaluation report, documentation and final demo. Roles: MediaPipe pipeline & features (member A), static & dynamic models (members A & B), sentence builder/TTS/auto-complete (member C), UI integration & documentation (member D), with weekly integration reviews.

14. Hardware and Software Requirements

Requirement	Minimum	Recommended
Processor	Dual/Quad-core CPU (Intel i3/i5 / Ryzen 3/5)	Quad-core+ CPU
RAM	4 GB	8 GB+
GPU	Not required	Optional NVIDIA GPU only for faster LSTM training
Camera	Built-in laptop webcam ($\geq$ 480p)	720p USB webcam
Storage	5 GB free	15 GB free (datasets + cached landmarks + models)
OS	Windows 10 / Ubuntu 20.04 / macOS	Ubuntu 22.04 / Windows 11
Software	Python 3.10, MediaPipe, OpenCV, scikit-learn/Keras, pytsx3, Tkinter/PyQt, Git	+ Jupyter, VS Code, Colab account
Audio	Working speakers/headphones (for TTS output)	—

Table 3.5: Hardware and software requirements.

15. Future Enhancements

- Expand the dynamic vocabulary toward full Pakistan Sign Language phrases; build a public PSL landmark dataset.
- Add facial-expression and body-pose cues (MediaPipe Holistic) for grammatically richer recognition.
- Move toward continuous sentence-level translation (sequence-to-sequence with a language model).
- Mobile app (MediaPipe + TFLite) for an always-available pocket interpreter.
- Two-way communication: speech-to-text plus animated/avatar sign output for the hearing-to-Deaf direction.
- Signer adaptation / few-shot personalisation; on-device continual learning from user corrections.
- Integration with video-call platforms for live captioning of signers.

16. Timeline / Gantt Chart

The project runs over a $\approx$ 16-week semester in three phases — foundation, modelling & development, and finalisation — with the weekly milestones below.

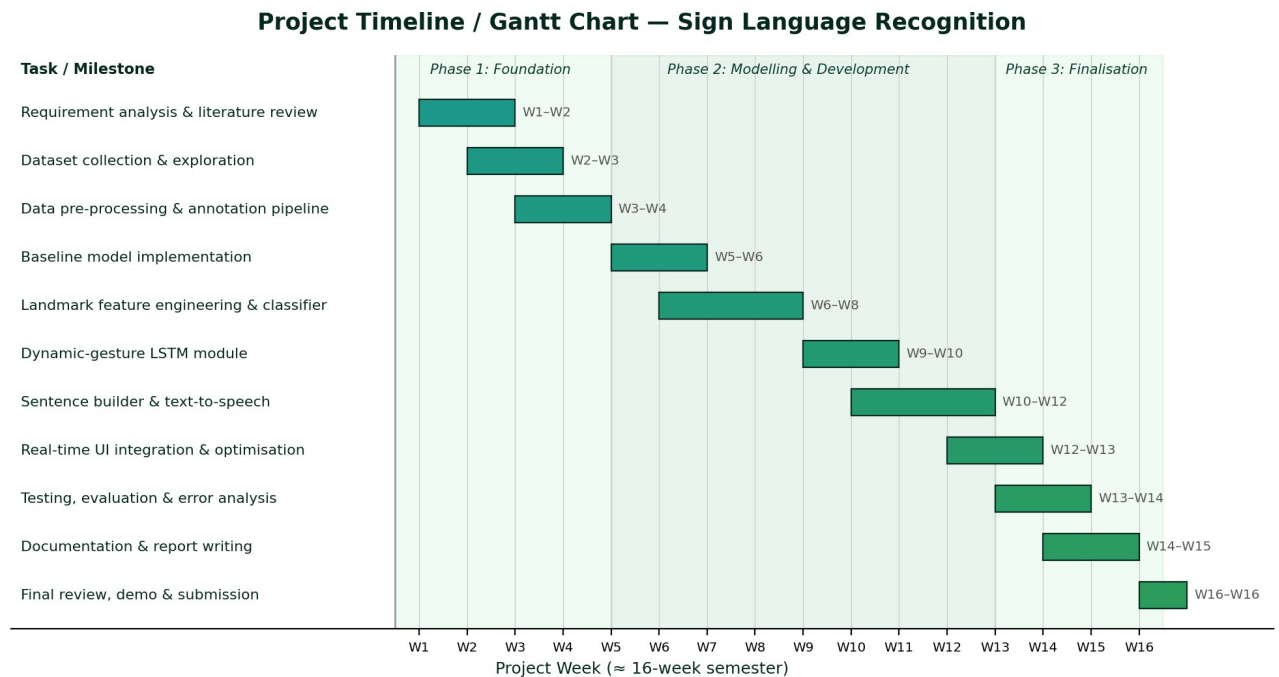


Figure 3.4: Project Gantt chart with weekly milestone breakdown.

Week(s)	Milestone / Deliverable
W1-W2	Requirement analysis, literature review, finalise architecture and tools
W2-W3	Download ASL/Sign-MNIST data; build MediaPipe landmark-extraction pipeline; EDA
W3-W4	Landmark normalisation & feature engineering; data-collection scripts; baseline static classifier
W5-W6	Collect custom alphabet/PSL data; train & tune static MLP/1-D CNN to target accuracy
W6-W8	Build dynamic-gesture recorder; assemble windowed sequence dataset; train LSTM/GRU
W9-W10	Temporal smoothing/debouncing; sentence builder with auto-complete
W10-W12	Integrate text-to-speech; build the real-time desktop UI (overlay, prediction, controls)
W12-W13	End-to-end integration; real-time optimisation (FPS, latency)
W13-W14	Testing with multiple signers; accuracy/confusion-matrix/latency evaluation
W14-W15	Documentation, user guide and final report writing

W16	Final review, demonstration and submission
------------	--

Table 3.6: Weekly milestone breakdown.

17. Conclusion

This proposal presented a Sign Language Recognition System that translates webcam hand-sign gestures into on-screen text and spoken output in real time. By building on Google's pretrained MediaPipe Hands to obtain compact, robust 21-point landmarks and classifying them with tiny neural networks — an MLP/1-D CNN for static signs and an LSTM/GRU for dynamic ones — the system achieves high accuracy while running comfortably on a GPU-free laptop. Beyond classification it delivers a complete assistive pipeline: temporal smoothing, sentence construction with auto-complete, and offline text-to-speech, with optional support for Pakistan Sign Language. The project is realistic for a four-member team within one semester, builds strong skills in real-time computer vision and sequence modelling, and produces a socially valuable communication aid with a clear roadmap toward continuous translation and mobile deployment.

18. References

References are listed in IEEE citation style; arXiv / dataset URLs are provided where applicable.

- [1] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann, "MediaPipe Hands: On-device real-time hand tracking," in CVPR Workshop on Computer Vision for Augmented and Virtual Reality, 2020. [Online]. Available: <https://arxiv.org/abs/2006.10214>
- [2] C. Lugaresi et al., "MediaPipe: A framework for building perception pipelines," arXiv:1906.08172, 2019.
- [3] D. Li, C. Rodriguez, X. Yu, and H. Li, "Word-level deep sign language recognition from video: A new large-scale dataset and methods comparison (WLASL)," in Proc. IEEE Winter Conf. on Applications of Computer Vision (WACV), 2020, pp. 1459–1469.
- [4] J. Carreira and A. Zisserman, "Quo vadis, action recognition? A new model and the Kinetics dataset (I3D)," in Proc. IEEE/CVF CVPR, 2017, pp. 6299–6308.
- [5] S. Yan, Y. Xiong, and D. Lin, "Spatial temporal graph convolutional networks for skeleton-based action recognition (ST-GCN)," in Proc. AAAI Conf. on Artificial Intelligence, 2018.
- [6] O. Koller, J. Forster, and H. Ney, "Continuous sign language recognition: Towards large vocabulary statistical recognition systems handling multiple signers," Computer Vision and Image Understanding, vol. 141, pp. 108–125, 2015.
- [7] N. C. Camgoz, S. Hadfield, O. Koller, H. Ney, and R. Bowden, "Neural sign language translation," in Proc. IEEE/CVF CVPR, 2018, pp. 7784–7793.
- [8] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.
- [9] IEEE Xplore, "Sign language recognition using deep learning — search results." [Online]. Available: <https://ieeexplore.ieee.org/search/searchresult.jsp?queryText=sign+language+recognition+deep+learning>
- [10] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.
- [11] F. Chollet et al., "Keras," 2015. [Online]. Available: <https://keras.io>
- [12] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

- [13]** N. M. Bhagat (Akash) and contributors, "pyttsx3 — offline text-to-speech conversion library for Python," documentation, 2023. [Online]. Available: <https://pyttsx3.readthedocs.io>
- [14]** Akash / Kaggle, "ASL Alphabet dataset." [Online]. Available: <https://www.kaggle.com/datasets/grassknoted/asl-alphabet>